Geometry Pipeline Development

Scope of the Geometry Contribution

The geometry pipeline contribution connects the project's room program and layout generation to usable two-dimensional and three-dimensional geometry in Rhino. Development began with an instructor-provided CPython test build that demonstrated a two-dimensional workflow. That starting point has since been refactored into a C# plug-in for Rhino 8 and extended to create three-dimensional room masses. The current implementation carries room information through spreadsheet reading, bubble generation, floor assignment, room layout, circulation generation, and document creation. Its central accomplishment is a connected workflow in which geometry and room identity remain associated as the representation changes. [1]

The work converts bubble relationships into walls, door openings, circulation outlines, and volumes for architectural inspection. Importing the generated design into Rhino means creating native document objects through the plug-in, without requiring an intermediate three-dimensional file.

Relationship to the Starting Build

The archived Grasshopper material preserves the earlier pipeline as six components: a spreadsheet reader, bubble solver, room tiler, corridor organizer, corridor-width stage, and door stage. Those stages establish the computational sequence inherited by the C# work. The archive also includes Grasshopper definitions, component scripts, screenshots, and handoff notes. Preserving those materials maintains a reference for the original behavior while allowing the current implementation to develop as a compiled plug-in. The historical component notes describe earlier work and results; they do not, by themselves, establish that the same results have been reproduced in the current build. [2]

The refactor preserves the general progression from program data to layout, while changing how the stages communicate and how users run them. The current project separates input parsing, layout decisions, circulation, volume creation, and Rhino document writing into named C# components. This structure makes the handoff between stages explicit and provides places to check room data before it becomes visible geometry. It also supports the three-dimensional extension without requiring an independent layout process for the volumes. [1]

Refactoring the Pipeline into C Sharp

Plug in Structure and Dependencies

The current plug-in targets the Windows version of .NET 7 and references RhinoCommon, the KangarooSolver library installed with Grasshopper, and ClosedXML version 0.105.0. The project

produces a Rhino plug-in assembly with an RHP extension during the build. The root-level Excel-to-bubble command is linked into that project, so it remains part of the active implementation despite being stored outside the main source folder. This arrangement gives the project a defined build entry point and retains the spreadsheet command in the same deployed plug-in as the floorplan generator. [1]

RhinoCommon provides the .NET interface used to interact with Rhino geometry and documents. McNeel identifies it as Rhino's cross-platform .NET plug-in SDK, although the present project is configured specifically for Windows. [3] The language change therefore places the application logic in a form that can call Rhino's document and geometry interfaces directly. Microsoft's C# documentation describes the language's object-oriented foundation and integration with .NET. [4] In this project, those capabilities are expressed through explicit room, placement, corridor, door, and result types rather than relying entirely on parallel component outputs.

Separation of Responsibilities

The generation command handles user interaction and coordinates the pipeline. Metadata parsing interprets the selected room circles. The layout engine assigns floors, forms clusters, and positions room rectangles. The circulation engine constructs corridor outlines and door placements. A document writer adds the results to Rhino, while separate helpers create room volumes, trim two-dimensional wall openings, and export the plan as SVG. Each stage therefore has a narrower responsibility than a single script that reads inputs, rearranges rooms, and creates every output object. [1]

Connecting Spreadsheet Data to Rhino Objects

Reading and Expanding the Room Program

The ExcelToBubbleDiagram command starts the current input path. It allows the user to choose a workbook and defaults to the ProgramInfo worksheet. The reader locates program fields including department number, program element, quantity, net square footage per unit, primary connections, color, and floor. Positive quantities and areas are checked before room instances are created. Quantity is expanded into separate nodes so a spreadsheet row describing several rooms becomes several identifiable geometric objects. ClosedXML supplies workbook access for this C# stage; its documentation identifies support for both XLSX and XLSM files. [1, 5]

Each room instance receives a unique identifier based on its department value and copy number, with a row-based fallback when the department is absent. Its bubble size is derived from net area after conversion into model units. The command uses Kangaroo constraints for bubble relaxation and creates colored geometry and labels in Rhino. Connection information influences the bubble arrangement. This

stage provides spatial input for the later room layout while preserving the numerical room requirement independently of the bubble's displayed dimensions. [1]

Metadata as the Handoff Contract

The important connection between the two commands is object user text. The bubble command attaches room identity, area, label, department, and floor preference to its output. It also records an adjacency reference when one is available. Generate2DFloorplan reads selected circles and reconstructs room records from that information. The circle center supplies a spatial preference, while the Area field controls the requested room area. Consequently, manually resizing a bubble does not replace the explicit area requirement. This distinction makes the input contract understandable and avoids treating a drawing edit as an undocumented program change. [1]

Validation checks protect that handoff. The parser requires a nonempty identifier and positive area, applies defaults for optional labels and departments, and interprets floor and door-width settings. It rejects duplicate identifiers and adjacency references that point outside the selected room set. These checks expose incomplete selections or inconsistent room information before the layout stage consumes them. Errors identify the affected objects or room identifiers, providing a more useful basis for correction than a later geometric failure.

The current interfaces also reveal limits that remain relevant to integration. The bubble command offers circles and squares, but the floorplan reader currently accepts circles. Shared-access information can be supplied through user text or fixtures, but the spreadsheet-to-bubble stamping routine does not currently populate the Shared field. It records one adjacency reference rather than preserving the full connection graph in the downstream room record. The completed handoff therefore supports a defined metadata contract, while complete transfer of every spreadsheet relationship remains a separate integration concern. [1]

Generating the Two Dimensional Layout

Floor Assignment and Room Grouping

The C# layout engine turns the parsed room set into floor assignments and rectangular placements. Floor capacity is calculated from the configured plate width, plate length, and efficiency. Preferred floor values guide placement, while shared-access rooms are grouped with their adjacency parents before assignment. Keeping these linked rooms together prevents the assignment stage from separating a room from the space through which it is intended to be accessed. When preferences cannot be met, the implementation can move groups and report the displacement. Floor-count limits provide a stopping condition rather than allowing uncontrolled growth. [1]

After floor assignment, the engine clusters rooms using features derived from bubble position, adjacency, and department. A fixed seed supports repeatable clustering for the same inputs and settings. The resulting groups are organized into rows around branch corridors and connected to trunk circulation. Within the packing sequence, shared rooms follow their parents, and related rooms are kept together when a row wraps. This carries relationship information beyond the initial bubble diagram into the physical arrangement of the plan.

Preserving Area During Room Shaping

Rectangular room dimensions are derived from the requested area and a preferred room depth. If the resulting rectangle exceeds the configured aspect-ratio limit, the dimensions are adjusted while retaining their area product. For example, changing a room's width to avoid an excessively narrow shape requires a corresponding change in depth. The layout therefore translates an area requirement into a practical rectangular representation without replacing that requirement with a rounded or approximate room size. The implementation also converts area using the square of the document's linear unit scale. [1]

The layout remains schematic. The floor plate is used as an area budget, and a diagnostic outline is created for comparison, but the engine does not guarantee that the completed arrangement fits within an arbitrary boundary. The archived handoff already identified why this issue is difficult: a later corridor organization stage can reposition rooms after an earlier stage has checked their fit. [2] The current C# work retains an explicit warning about containment rather than presenting the area calculation as proof that the final arrangement fits geometrically.

Floors are displayed beside one another at the plan level, so their numbers describe assignment rather than stacked building elevations. A plan view showing the separated floors, labels, and corridor organization would illustrate this output without being necessary to interpret it. [1]

INSERT SCREENSHOT HERE

Creating Circulation and Readable Plan Geometry

Corridor Outlines and Door Placement

The circulation stage converts corridor centerlines into closed ribbon outlines with the assigned widths. Branch and trunk corridors are represented separately. Trunk width increases with the number of rooms served according to a configurable formula, subject to a maximum width. This creates a visible distinction between local access and circulation serving larger room groups. The formula is a project layout rule; the generated widths do not constitute a demonstrated assessment of building-code or accessibility compliance. [1]

Door generation uses the placed room geometry rather than the original bubble circles. For ordinary rooms, the engine chooses an access edge in relation to nearby corridor segments. For a shared-access room, it first attempts to identify a shared wall with the adjacency parent. When that relationship cannot be represented by a suitable shared edge, the engine generates a corridor door and records a warning. This behavior retains a visible access representation while exposing the relationship that could not be satisfied.

Each door is drawn with a leaf line in the open position and a quarter-circle swing arc. Wall-opening logic removes the corresponding interval from the room's two-dimensional wall segments. A plan therefore contains an actual linework gap at the door location, rather than a swing symbol laid over an uninterrupted room rectangle. The distinction is useful during review because room enclosure and access can be read together. The widths are constrained by the available edge length, and walls that cannot accommodate a usable generated opening produce warnings. [1]

Organizing the Output in Rhino

The document writer places geometry beneath a Floorplan layer hierarchy, with separate floor branches and layers for room walls, room masses, stairs, labels, corridors, doors, and diagnostics. Branch and trunk corridor outlines occupy separate sublayers. Room colors are preserved, and each floor's generated objects are grouped. This organization allows the reviewer to isolate a representation, compare circulation with room boundaries, or hide the three-dimensional masses to inspect the plan underneath. [1]

Object metadata remains available after generation. Room outputs carry identifiers, requested area, department, assigned floor, cluster, shared-access status, and adjacency information. Door objects retain their associated room identifier and door kind. Corridor objects store width and service information. These attributes make the generated document more useful than an unstructured collection of lines because the geometry can be traced back to the room records that produced it. A close plan view with the layer panel and one selected room's user text would make that connection visible without being necessary to understand the workflow.

INSERT SCREENSHOT HERE

The command also places its Rhino document changes within an undo record. One Undo can reverse the geometry created by a generation run. This supports iterative inspection when settings or selections need adjustment. The SVG written to disk is handled separately, so document undo does not restore an earlier exported file. [1]

Extending Room Footprints into Three Dimensions

Room Mass Creation

The three-dimensional extension builds capped room masses from the same complete rectangular footprints used by the plan. Each ordinary room receives a vertical extrusion with a default height of 14 feet. The resulting object is checked for validity and solidity before it is added to Rhino. Because the footprint comes from the established room placement, the mass retains the room's position and requested plan area. The extension adds a volumetric representation without introducing a second source of layout decisions. [1]

Room masses are written to the Rooms3D layer for each floor. Their attributes preserve room identity and color and include HeightFeet, which records the height used to generate the object. The room's two-dimensional walls and label are still created. This allows the same generated arrangement to be reviewed in plan and in a shaded perspective view. Switching layer visibility provides a straightforward way to compare the representations while retaining the room information needed to identify each mass.

Height Settings and Unit Conversion

Height handling is separated from the user interface. The volume settings provide a default height and an optional dictionary of overrides indexed by room identifier. Override lookup is case insensitive, and heights must be finite and greater than zero. The current command uses the default settings; a dedicated height-editing interface and Excel-based height input have not been added. The settings structure nevertheless establishes a specific connection point for a later interface without requiring the extrusion logic to be rewritten. [1]

The height is converted from feet into the active Rhino document's units before extrusion. The same scale is used elsewhere for linear geometry, while areas require its square. This distinction matters when the document uses millimeters: a height of 14 feet must become 4,267.2 millimeters rather than 14 model units. The implementation's unit conversion therefore connects program values expressed in feet and square feet to geometry drawn at the intended physical size.

Scope of the Current Three Dimensional Output

Staircases and corridors remain flat. Stair classification uses an explicit RoomType value when supplied and otherwise recognizes stair-related labels. Ordinary words such as "Upstairs office" remain room labels rather than triggering staircase classification. A staircase is represented by a closed planar footprint, not by modeled flights, risers, or landings. Floors also remain side by side rather than being stacked at building elevations. [1]

The room masses represent full volumes and do not contain three-dimensional doorway cutouts. The plan's wall gaps and swing symbols remain separate two-dimensional information. The present output is therefore suitable for inspecting room size, grouping, and basic massing, while detailed wall assemblies and building construction remain outside the implemented geometry. A shaded view comparing ordinary room masses with flat circulation and stair footprints would illustrate this distinction clearly.

INSERT SCREENSHOT HERE

Export and Verification Support

A Separate Two Dimensional Export

The pipeline also exports the current plan to floorplan.svg in the build root. The export includes the generated floors, room wall segments, corridor outlines, door leaves, swing arcs, and labels. Three-dimensional room masses and diagnostic geometry are excluded. This provides a portable plan representation for inspection outside Rhino while keeping the modeling handoff within the native document. The export uses the same generated result and wall-opening calculations as the Rhino output, reducing the chance that the two plan representations describe different room arrangements. [1]

The SVG helper reflects the vertical coordinate for the SVG coordinate system, calculates drawing bounds, and adds margins. Labels are emitted through an XML structure so characters such as angle brackets and ampersands can be preserved correctly. Each generation replaces the existing SVG with the latest output rather than accumulating earlier plans. Export errors are reported independently after document creation, allowing successfully generated Rhino geometry to remain available when the disk export fails. [1]

Checks for Data and Geometry

The current project includes a standalone test entry point, an in-Rhino self-test command, a sample-input fixture command, and a Rhino smoke-test script. The managed checks cover metadata interpretation and validation, stair classification, room-height settings, corridor-width calculations, floor assignment, repeatable clustering, exact shaped areas, and the ordering of shared-access chains. This separates checks of room and layout logic from checks that require Rhino's native geometry environment. [1]

The geometry checks extend into circulation, shared walls, wall openings, door swings, room masses, and SVG content. Volume assertions compare the generated height and volume against the requested area and configured height. They exercise both feet and millimeter scale factors and verify that stairs remain flat. SVG assertions examine labels, corridor counts, floor groups, door arc paths, drawing bounds, coordinate reflection, and replacement of earlier output. These are implemented checks with specific expected relationships; their presence does not establish that every current scenario has passed a recorded test run.

Document-level bake tests inspect the objects and layers produced by the writer. They check that ordinary rooms produce masses, mass metadata includes the default height, and the staircase output remains a closed flat curve. The fixture command creates tagged circles without requiring Excel, allowing the floorplan stages to be investigated independently of workbook reading. The smoke-test script then exercises plug-in loading, fixture generation, selection, and command execution inside Rhino. Together, these tools give the project a way to distinguish data errors, layout errors, native geometry errors, and document-writing errors. [1]

Current Implementation Boundaries

Several integration details remain visible in the current implementation. The spreadsheet command relaxes bubbles using floor-based groups, while the archived handoff describes a later global-cloud approach. The downstream floorplan engine performs its own floor assignment from preferences and capacity. These stages should therefore be described according to their current C# behavior rather than assuming that every archived Python change was transferred unchanged. Likewise, shared-access behavior is implemented downstream, but its automatic population from spreadsheet data is incomplete. [1, 2]

The project guides document building and installing the RHP, selecting the workbook, and generating a plan from tagged circles. Rhino must restart after rebuilding a loaded assembly. Grasshopper remains a runtime dependency because the bubble command uses its installed KangarooSolver library. This sequence supports continued team testing and integration. [1]

References

[1] Project development team. Current implementation and guides, reviewed October 26, 2026. Retrieval: Rhino2DGeometry source and README; Rhino2DGeometry.Tests; ExcelToBubbleDiagramCommand.cs. Local archive: C:/Users/Kevin/Documents/Senior Design/Week 1 Build.

[2] Project handoff archive. Generative Floor Plan Pipeline Technical Handoff. Author unidentified. Retrieval: grasshopper_build/ClaudeHandoff/Floorplan_Pipeline_Handoff.md in the archive cited in [1].

[3] Steve Baer, Robert McNeel & Associates. What is RhinoCommon? Rhino Developer, updated October 7, 2024. https://developer.rhino3d.com/guides/rhinocommon/what-is-rhinocommon/

[4] Microsoft. A tour of the C# language. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/overview

[5] ClosedXML contributors. ClosedXML documentation. https://docs.closedxml.io/en/latest/. External references accessed October 26, 2026.